

part_2

May 26, 2026

```
[1]: import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import numpy as np
from skimage.transform import resize
import random
import os
from PIL import Image
import pandas as pd
import math
from matplotlib.image import imread

print("Done importing libraries\n")
```

Done importing libraries

0.0.1 Preprocessing: Linear Contrast Stretching

(Applied after the Median and Gaussian filters)

```
[ ]: def linear_contrast_stretching(image):
    img = image.copy()
    #Store the min and max intensity in the grayscale dog image.
    min_intensity = np.min(img)
    max_intensity = np.max(img)

    linear_contrast_stretched_image = img.copy()
    denominator = max_intensity - min_intensity
    if denominator != 0: #We can't divide by zero!
        return (img - min_intensity) * (255 / denominator) #Apply the linear_
        ↪ contrast stretching formula.
    else:
        print("Cannot apply linear contrast stretching because all pixel_
        ↪ intensities are the same. Please try with a different image.")

print("Defined linear contrast stretching function")
```

0.0.2 Preprocessing: Linear Contrast Stretching on Median_Gaussian_Filtered_Images

```
[ ]: if not os.path.exists("contrast_stretched_median_gaussian_images"):
    os.makedirs("contrast_stretched_median_gaussian_images")
    print(f"Created folder: contrast_stretched_median_gaussian_images")
else:
    print(f"Folder contrast_stretched_median_gaussian_images already exists.")

print("Linear Contrast Stretching 10 Images\n")
for i in range(1, 11):
    print("Enhancing Image " + str(i))
    #gaussian filter -> save the result to median_gaussian_filtered_images_
    ↪folder
    current_image = imread("median_gaussian_filtered_images/
    ↪median_gaussian_img_" + str(i) + ".jpg")
    stretch_result = linear_contrast_stretching(current_image)
    res_img = Image.fromarray(stretch_result)
    res_img.convert("L").save(f"contrast_stretched_median_gaussian_images/
    ↪contrast_stretched_median_gaussian_img_{i}.jpg")
```

0.0.3 Define Otsu Thresholding Function

```
[ ]: def otsu(image):
    # First compute the histogram
    histogram, bin_edges = np.histogram(image.flatten(), bins=256, range=(0,
    ↪255))

    probabilities = histogram / image.size

    # t is the intensity values (0 to 255)
    # (t) aka prob_class1
    #prob_class2 = 1 - prob_class1
    cumulative_sum = np.cumsum(probabilities)
    # (t) = i * p(i)
    cumulative_mean = np.cumsum(probabilities * np.arange(256))

    #last value of cumulative mean
    global_mean_intensity = cumulative_mean[-1]

    denominator = cumulative_sum * (1 - cumulative_sum)

    # Only compute variance where denominator is non-zero. (Can't divide by 0)
    # this is a mask
    valid = denominator > 0
    between_class_variance = np.zeros(256)
```

```

        between_class_variance[valid] = (global_mean_intensity *
↪cumulative_sum[valid] - cumulative_mean[valid]) ** 2 / \
                                denominator[valid]

    threshold = np.argmax(between_class_variance)
    return threshold

```

0.0.4 Functions to apply Thresholds to binarize an image

```

[ ]: def apply_threshold(image, threshold):
    img = image.copy()
    one = img >= threshold
    zero = img < threshold
    img[one] = 255
    img[zero] = 0
    return img

```

```

[ ]: def apply_multiple_thresholds(image, thresholds):
    result = np.zeros_like(image)
    thresholds = sorted(thresholds)
    thresholds.append(256) #Upper bound for the last region
    lower = 0
    for i in range(len(thresholds)):
        upper = thresholds[i]
        mask = (image >= lower) & (image < upper)
        result[mask] = i * (255 // (len(thresholds) - 1))
        lower = upper

    return result

```

0.0.5 Helper function to plot histogram of an image

```

[ ]: def plot_histogram(image, filename, image_number):
    plt.figure()
    plt.hist(image.flatten(), bins=256, range=(0, 255), color="steelblue")
    plt.xlabel("Intensity")
    plt.ylabel("Count")

    plt.title(f"Grayscale Histogram - Image {image_number}")
    plt.savefig(filename)
    plt.show()

```

0.0.6 Generate and save histograms for each image

```
[ ]: if not os.path.exists("histograms_of_filtered_images"):
    os.makedirs("histograms_of_filtered_images")
    print(f"Created folder: histograms_of_filtered_images")
else:
    print(f"Folder histograms_of_filtered_images already exists.")

print("Creating 10 Histograms\n")
for i in range(1, 11):
    print("Creating Histogram for Image " + str(i))
    contrast_stretched_filepath = "contrast_stretched_median_gaussian_images"
    current_image = imread(f"{contrast_stretched_filepath}/
↪contrast_stretched_median_gaussian_img_{i}.jpg")
    plot_histogram(current_image, f"histograms_of_filtered_images/histogram_{i}.
↪png", i)

print("Done creating histograms!")
```

0.0.7 Apply Otsu Thresholding to each image

```
[ ]: otsu_thresholds = []
image_dir = "contrast_stretched_median_gaussian_images"
image_name = "contrast_stretched_median_gaussian_img_"

if not os.path.exists("otsu_images"):
    os.makedirs("otsu_images")
    print(f"Created folder: otsu_images")
else:
    print(f"Folder otsu_images already exists.")

print("Otsu Thresholding 10 Images\n")
for i in range(1, 11):
    print("Thresholding Image " + str(i))
    #otsu thresholding -> save the result to otsu_images folder
    current_image = imread(f"{image_dir}/{image_name}{i}.jpg")
    t = otsu(current_image)
    otsu_thresholds.append(int(t))
    binary_image = (current_image > t).astype(np.uint8) * 255
    res_img = Image.fromarray(binary_image)
    res_img.convert("L").save(f"otsu_images/otsu_img_{i}.jpg")

print("\nThe Otsu thresholds for the 10 images are: ", otsu_thresholds)
```

0.0.8 Manual Thresholding (Single & Multi level)

```
[ ]: multiple_thresholds = [5,45]
single_threshold = int(np.average(otsu_thresholds))

print("The single threshold is the average of the Otsu thresholds: ",
      ↪single_threshold, "\n")
image_dir = "contrast_stretched_median_gaussian_images"
image_name = "contrast_stretched_median_gaussian_img_"

if not os.path.exists("single_threshold_images"):
    os.makedirs("single_threshold_images")
    print(f"Created folder: single_threshold_images")
else:
    print(f"Folder single_threshold_images already exists.")

if not os.path.exists("multiple_threshold_images"):
    os.makedirs("multiple_threshold_images")
    print(f"Created folder: multiple_threshold_images")
else:
    print(f"Folder multiple_threshold_images already exists.")

print("\nManually Thresholding 10 Images\n")
for i in range(1, 11):
    print("Thresholding Image " + str(i))
    #manual thresholding -> save the result to single_threshold_images folder
    current_image = imread(f"{image_dir}/{image_name}{i}.jpg")
    single_result = apply_threshold(current_image, single_threshold)
    multiple_result = apply_multiple_thresholds(current_image,
    ↪multiple_thresholds)

    single_result = apply_threshold(current_image, single_threshold)
    Image.fromarray(single_result).convert("L").save(f"single_threshold_images/
    ↪single_img_{i}.jpg")

    multiple_result = apply_multiple_thresholds(current_image,
    ↪multiple_thresholds)
    Image.fromarray(multiple_result).convert("L").
    ↪save(f"multiple_threshold_images/multiple_img_{i}.jpg")
```

0.1 Student 2

0.1.1 Part A: Adaptive Local Thresholding Function

```
[2]: def adaptive_local_threshold(image, window_size, k, M):
    img = image.copy()
    if(np.max(img) <= 1.0):
        img = img * 255.0

    pad = window_size // 2
    padded = np.pad(img, pad, mode='constant', constant_values=0)
    result = np.zeros_like(img)

    for row in range(img.shape[0]):
        for col in range(img.shape[1]):
            region = padded[row:row+window_size, col:col+window_size]
            local_mean = np.mean(region)
            local_max = np.max(region)
            local_min = np.min(region)

            threshold = k * (local_mean + ((local_max - local_min) / M))

            if img[row, col] >= threshold:
                result[row, col] = 255
            else:
                result[row, col] = 0

    return result
```

0.1.2 Part B: Sauvola Adaptive Thresholding Function

```
[3]: def sauvola_threshold(image, window_size, k, R):
    img = image.copy()
    if(np.max(img) <= 1.0):
        img = img * 255.0

    pad = window_size // 2
    padded = np.pad(img, pad, mode='constant', constant_values=0)
    result = np.zeros_like(img)

    for row in range(img.shape[0]):
        for col in range(img.shape[1]):
            region = padded[row:row+window_size, col:col+window_size]
            local_mean = np.mean(region)
            local_std = np.std(region)

            threshold = local_mean * (1 + k * ((local_std / R) - 1))
```

```

        if img[row, col] >= threshold:
            result[row, col] = 255
        else:
            result[row, col] = 0

    return result

```

0.1.3 Helper functions for comparison

```

[4]: def get_foreground_area_percent(binary_image):
    foreground_pixels = np.sum(binary_image > 0)
    total_pixels = binary_image.shape[0] * binary_image.shape[1]
    return (foreground_pixels / total_pixels) * 100

def get_foreground_background_gap(image, binary_image):
    img = image.copy()
    if(np.max(img) <= 1.0):
        img = img * 255.0

    foreground = img[binary_image > 0]
    background = img[binary_image == 0]

    if len(foreground) == 0 or len(background) == 0:
        return 0

    return np.mean(foreground) - np.mean(background)

def k_to_filename(k):
    return str(k).replace('.', '_')

def plot_student2_k_comparison(image_number, adaptive_results, sauvola_results,
    ↪adaptive_k_values, sauvola_k_values):
    plt.figure(figsize=(14, 8))

    plot_number = 1
    for k in adaptive_k_values:
        plt.subplot(2, 3, plot_number)
        plt.imshow(adaptive_results[k], cmap='gray')
        plt.title('Adaptive Local k=' + str(k))
        plt.axis('off')
        plot_number += 1

    for k in sauvola_k_values:
        plt.subplot(2, 3, plot_number)

```

```

plt.imshow(sauvola_results[k], cmap='gray')
plt.title('Sauvola k=' + str(k))
plt.axis('off')
plot_number += 1

plt.suptitle('k-value comparison - image ' + str(image_number))
plt.tight_layout()
plt.savefig('part_2_student_2_outputs/student2_comparison_images/
↳comparison_' + str(image_number) + '.png', dpi=200, bbox_inches='tight')
plt.close()

```

0.1.4 Apply methods to each image

```

[7]: adaptive_k_values = [0.9, 1.0, 1.1]
sauvola_k_values = [0.2, 0.35, 0.5]
sauvola_R = 128
window_size = 51
M = 255

student2_results = []
image_dir = "contrast_stretched_median_gaussian_images"
image_name = "contrast_stretched_median_gaussian_img_"

if not os.path.exists("part_2_student_2_outputs"):
    os.makedirs("part_2_student_2_outputs")

if not os.path.exists("part_2_student_2_outputs/adaptive_local_images"):
    os.makedirs("part_2_student_2_outputs/adaptive_local_images")
    print(f"Created folder: adaptive_local_images")
else:
    print(f"Folder adaptive_local_images already exists.")

if not os.path.exists("part_2_student_2_outputs/sauvola_images"):
    os.makedirs("part_2_student_2_outputs/sauvola_images")
    print(f"Created folder: sauvola_images")
else:
    print(f"Folder sauvola_images already exists.")

if not os.path.exists("part_2_student_2_outputs/student2_comparison_images"):
    os.makedirs("part_2_student_2_outputs/student2_comparison_images")
    print(f"Created folder: student2_comparison_images")
else:
    print(f"Folder student2_comparison_images already exists.")

print("Thresholding 10 Images")
for i in range(1, 11):
    print("Thresholding Image " + str(i))

```



```

current_image = imread(f"{image_dir}/{image_name}{i}.jpg")
if np.max(current_image) <= 1.0:
    current_image = current_image * 255.0

adaptive_results = {}
sauvola_results = {}

for k in adaptive_k_values:
    adaptive_result = adaptive_local_threshold(current_image, window_size,
↪k, M)
    adaptive_results[k] = adaptive_result
    output_path = "part_2_student_2_outputs/adaptive_local_images/
↪adaptive_local_img_" + str(i) + "_k_" + k_to_filename(k) + ".jpg"
    Image.fromarray(adaptive_result.astype(np.uint8)).convert("L").
↪save(output_path)

    student2_results.append({
        "image_#": i,
        "method": "Adaptive Local Thresholding",
        "k": k,
        "foreground_area_percent":
↪get_foreground_area_percent(adaptive_result),
        "foreground_background_gap":
↪get_foreground_background_gap(current_image, adaptive_result)
    })

for k in sauvola_k_values:
    sauvola_result = sauvola_threshold(current_image, window_size, k,
↪sauvola_R)
    sauvola_results[k] = sauvola_result
    output_path = "part_2_student_2_outputs/sauvola_images/sauvola_img_" +
↪str(i) + "_k_" + k_to_filename(k) + ".jpg"
    Image.fromarray(sauvola_result.astype(np.uint8)).convert("L").
↪save(output_path)

    student2_results.append({
        "image_#": i,
        "method": "Sauvola Thresholding",
        "k": k,
        "foreground_area_percent":
↪get_foreground_area_percent(sauvola_result),
        "foreground_background_gap":
↪get_foreground_background_gap(current_image, sauvola_result)
    })

```

```

    plot_student2_k_comparison(i, adaptive_results, sauvola_results,
↪adaptive_k_values, sauvola_k_values)

student2_results_df = pd.DataFrame(student2_results)
print("Done creating thresholded images and comparison panels")

```

Folder adaptive_local_images already exists.
Folder sauvola_images already exists.
Folder student2_comparison_images already exists.
Thresholding 10 Images
Thresholding Image 1
Thresholding Image 2
Thresholding Image 3
Thresholding Image 4
Thresholding Image 5
Thresholding Image 6
Thresholding Image 7
Thresholding Image 8
Thresholding Image 9
Thresholding Image 10
Done creating thresholded images and comparison panels

0.1.5 Analysis table

```

[6]: student2_summary = (
    student2_results_df
    .groupby(["method", "k"])
    .agg(
        average_foreground_area_percent=("foreground_area_percent", "mean"),
        average_foreground_background_gap=("foreground_background_gap", "mean")
    )
    .reset_index()
)
student2_summary

```

```

[6]:
   method      k  average_foreground_area_percent \
0  Adaptive Local Thresholding  0.90          54.927559
1  Adaptive Local Thresholding  1.00          45.061951
2  Adaptive Local Thresholding  1.10          36.527977
3      Sauvola Thresholding  0.20          60.702057
4      Sauvola Thresholding  0.35          66.155319
5      Sauvola Thresholding  0.50          69.192734

   average_foreground_background_gap

```

0	31.939983
1	21.410558
2	10.454141
3	36.066773
4	40.253098
5	41.921576